What works:

Currently, the API is authenticated and data retrieval is possible from CollegeFootballData.org API. Data is stored locally and in AWS S3. Project directory is also setup. The project directory is structured to separate ingestion, processing, and configuration concerns. API and AWS credentials are secure, utilizing the python-dotenv library and a .gitignore file to prevent keys from being committed to the public repo.

Data Acquisition Test:

API response when
test_year = 2025 test_week = 1 test_team = "Georgia"

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   Python
                                                                                             Python
        "away": "Marshall",
        "period": 3,
        "clock": {
            "minutes": 13,
            "seconds": 13
        },
        "offenseTimeouts": 3,
        "defenseTimeouts": 3,
        "yardline": 68,
        "yardsToGoal": 68,
        "down": 2,
        "distance": 15,
        "yardsGained": -1,
        "scoring": false,
        "playType": "Rush",
        "playText": "Antwan Roberts run for a loss of 1 yard to the MRSH 31",
        "ppa": -0.3709996187496536,
        "wallclock": "2025-08-30T21:49:35.000Z"
    },
    {
        "gameId": 401752669,
        "driveId": "40175266917",
        "id": "401752669103868610",
        "driveNumber": 17,
        "playNumber": 6,
        "offense": "Marshall",
        "offenseConference": "Sun Belt",
        "offenseScore": 0,
        "defense": "Georgia",
        "defenseConference": "SEC",
        "defenseScore": 31,
        "home": "Georgia",
        "away": "Marshall",
        "period": 3,
rd                  johnlroehsler (2 minutes ago)   Ln 32, Col 27   Spaces: 4   UTF-8   LF   { } Python   Python 3.9.6 (xcode)
```

Successful upload to S3:

```
/Users/johnroehsler/Library/Python/3.9/lib/python/site-packages/boto3/compat.py:89: PythonDeprecationWarning: B
oto3 will no longer support Python 3.9 starting April 29, 2026. To continue receiving service updates, bug fixe
s, and security updates please upgrade to Python 3.10 or later. More information can be found here: https://aws
.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
Uploading to S3: s3://s3-college-football-db/raw/year=2025/week=1/Georgia_plays.json
Upload complete.
johnroehsler@Mac-Mini collegeFootball %
d                   Ln 59, Col 1 (64 selected)   Spaces: 4   UTF-8   LF   { } Python   Python 3.9.6 (xcode)
```

Successful retrieval from S3:

```
        "playNumber": 3,
        "offense": "Marshall",
        "offenseConference": "Sun Belt",
        "offenseScore": 7,
        "defense": "Georgia",
        "defenseConference": "SEC",
        "defenseScore": 45,
        "home": "Georgia",
        "away": "Marshall",
        "period": 4,
        "clock": {
            "minutes": 0,
            "seconds": 0
        },
        "offenseTimeouts": 2,
        "defenseTimeouts": 2,
        "yardline": 76,
        "yardsToGoal": 76,
        "down": 3,
        "distance": 8,
        "yardsGained": 3,
        "scoring": false,
        "playType": "Rush",
        "playText": "Judah Price run for 3 yds to the MRSH 27",
        "ppa": -0.08245820092417055,
        "wallclock": "2025-08-30T23:03:46.000Z"
    }
]
----------------------------------------
Total plays retrieved: 173
johnroehsler@Mac-Mini collegeFootball %
```


Challenges Encountered:

There weren't too many challenges, but the hardest part was getting the API to work. The documentation online wasn't super clear, so I had to mess around with it to get exactly what I wanted. I also had some trouble with Git, but that was mainly due to my lack of experience with it.

Changes from M1:

The only change from M1 is selecting S3 for my storage layer. Previously I said HDFS or S3.

Next Steps:

I am going to, Initialize the Apache Spark environment to handle the data transformations, develop a Spark job to read the raw JSON files from the S3 bucket, enforce the data schema using Spark DataFrames , write the output back to storage as Parquet files with Snappy compression, implement the MapReduce pattern using Spark to group plays by team_id and game_id, and do benchmarking.

Next Steps:

I am currently on track.